SOFTWARE REQUIREMENTS SPECIFICATION

Inventory Service

Product catalog, stock transactions, reservations, warehouses, reporting, and integrations.

PROJECT NAME	**Inventory (inventory-service)**
DOCUMENT TITLE	**Software Requirements Specification (SRS)**
VERSION	**1.0**
DATE	**07 August 2026**
PREPARED BY	**K. Sai Chaitanya**
REVIEWED BY	**TBD**
APPROVED BY	**TBD**

REVISION HISTORY

VERSION	DATE	AUTHOR	CHANGES
1.0	07-Aug-2026	K. Sai Chaitanya	Initial version

Purpose, background, and objectives

1.1 Purpose

This document describes the functional and non-functional requirements for the Inventory backend service (`inventory-service`) supporting product catalog management, stock transactions, reservations, warehouses, reporting, and upstream integrations.

1.2 Project Background

Repository: `com.fawnix.inventory` (Spring Boot). Key modules: `products`, `transactions`, `reservations`, `warehouses`, `security`. Database migrations live under `src/main/resources/db/migration` (V1–V6).

1.3 Business Problem

Provide reliable, auditable stock visibility and reservation/fulfillment flows for warehouse and upstream systems (ERP/POS), preventing oversell and enabling reconciliation.

1.4 Objectives

- Accurate stock tracking through full CRUD and transaction recording
- Reservation and fulfillment flows that prevent double-commit
- Secure, auditable REST APIs for integrations
- Reasonable performance and operational reliability

1.5 Intended Audience

Product owners, developers, QA, DevOps, and integrators.

1.6 Definitions / Abbreviations

Term	Definition
SKU	Stock Keeping Unit

Term	Definition
API	Application Programming Interface
JWT	JSON Web Token
ERP/POS	Enterprise Resource Planning / Point of Sale
FR-INV	Functional Requirement — Inventory module
NFR	Non-Functional Requirement

2 — SCOPE

What's in, what's out, what's assumed

2.1 In Scope

- Product / catalog management — `src/main/java/.../products`
- Stock transactions: inbound, outbound, adjustments — `transactions`
- Inventory reservations: validate, reserve, fulfill — `reservations`
- Warehouse management — `warehouses`
- REST APIs under `/inventory` and `/internal/inventory/reservations`
- JWT-based auth and internal-service filter — `security`
- PostgreSQL database with migrations

2.2 Out of Scope

- Offline-first mobile clients
- Advanced forecasting or machine learning
- Payment flows
- Multi-region active-active replication (beyond standard DB replicas)

2.3 Assumptions

- Consumers access the service over HTTPS
- Internal reservation endpoints are called only by trusted internal systems
- PostgreSQL is available and migrations are applied on deployment

2.4 Dependencies

- Java / Spring Boot runtime
- PostgreSQL database
- SMTP (optional) for notifications

- Upstream ERP/POS for integrations

Product, roles, environment, architecture

3.1 Product Overview

A backend service exposing JSON REST endpoints to manage products, stock, reservations, and warehouses. Implements audit logging via the `stock_transactions` table and enforces authentication via JWT and `InternalServiceAuthFilter`.

3.2 User Roles

Warehouse operator, Inventory manager, Admin, Auditor, Internal service (machine client).

3.3 Operating Environment

- **Server:** Linux or Windows host
- **Database:** PostgreSQL
- **Clients:** web UI or other services via REST

3.4 System Architecture Overview

- Spring Boot layered architecture: controllers → services → repositories (JPA) → database
- Security filters: `JwtAuthenticationFilter`, `InternalServiceAuthFilter`
- Database migrations under `resources/db/migration`

3.5 External Systems

- ERP/POS (via REST/webhooks)
- Authentication provider (JWT issuer; may be an internal module)
- Email/SMS gateway for alerts

FR-INV-001 through FR-INV-010

Each requirement below follows the same structure: actors, preconditions, inputs, process, output, error behaviour, and priority.

FR-INV-001 User Login — HIGH

DESCRIPTION	System shall allow registered users to authenticate via JWT.
ACTORS	Admin, Manager, Employee
PRECONDITIONS	User account exists and is active.
INPUTS	Username/email, password
PROCESS	Validate credentials, generate JWT
OUTPUT	JWT token and user details
ERROR	Invalid credentials → `401 Unauthorized`

FR-INV-002 User Management — HIGH

DESCRIPTION	Admins manage users and roles.
ACTORS	Admin
PRECONDITIONS	Admin authenticated
INPUTS	User details, role
PROCESS	Create, update, or deactivate a user
OUTPUT	User resource
ERROR	Duplicate email → `409 Conflict`

FR-INV-003 Product Management (FR-INV-PROD-001) — HIGH

DESCRIPTION	CRUD product master records — id, sku, name, description, category, unit, price tiers, stockQty, reservedQty, status.
ACTORS	Admin, Manager
PRECONDITIONS	Authenticated with permitting role
INPUTS	Product DTO — `ProductDtos.java`
PROCESS	Validate, persist; optional seed data via V4 migration
OUTPUT	Product resource
ERROR	Invalid fields → `400 Bad Request`

FR-INV-004 Stock Transactions (FR-INV-TXN-001) HIGH

DESCRIPTION	Record inbound, outbound, and adjustment transactions.
ACTORS	Warehouse operator, Internal services
PRECONDITIONS	Product exists
INPUTS	Transaction DTO (`TransactionDtos.java`): productId, txnType, qty, reference, txnDate
PROCESS	Persist `StockTransactionEntity`; update `product.stockQty`; maintain transaction indexes (`idx_stock_txn_product`, `idx_stock_txn_date`)
OUTPUT	Transaction id and updated stock level
ERROR	Insufficient stock for outbound → `400 Bad Request`

FR-INV-005 Inventory Reservation (FR-INV-RES-001) HIGH

DESCRIPTION	Reserve inventory for orders; support validate, reserve, and fulfill stages.
ACTORS	Internal services (order system), Warehouse
PRECONDITIONS	Product exists; stockQty accounts for reservedQty
INPUTS	List of `{productId, quantity}`
PROCESS	Validate availability; reserve by incrementing reservedQty and creating reservation records; fulfill reduces reservedQty and stockQty and writes transactions

OUTPUT	Reservation response with per-line success/failure
ERROR	Insufficient stock → clear message; no partial commit unless configured

FR-INV-006 Warehouse Management (FR-INV-WH-001) MEDIUM

DESCRIPTION	CRUD warehouses and query capabilities.
ACTORS	Admin, Manager
PRECONDITIONS	Authenticated
INPUTS	Warehouse DTO — `WarehouseDtos.java`
OUTPUT	Warehouse resource

FR-INV-007 Inventory Overview / Reporting (FR-INV-RPT-001) MEDIUM

DESCRIPTION	Provide inventory overview and movement history.
ACTORS	Manager, Auditor, Internal services
INPUTS	Query parameters — sku, location, date range
PROCESS	Aggregate via product and stock-transaction tables
OUTPUT	`InventoryOverviewDtos`; CSV/PDF export

FR-INV-008 Audit Logging & Error Handling (FR-INV-AUD-001) HIGH

DESCRIPTION	All stock changes and reservation actions shall be logged with user, timestamp, and reason.
ACTORS	System
PRECONDITIONS	Database available
OUTPUT	Entries in `stock_transactions` and application logs; exceptions returned as `ApiErrorResponse`

FR-INV-009 API Rate Limiting & API Keys (FR-INV-API-001) LOW (RECOMMENDED)

DESCRIPTION	External APIs require API keys; rate limiting is configurable.
ACTORS	External integrators

FR-INV-010 Barcode Scanning Support (FR-INV-HW-001) LOW

DESCRIPTION	Input endpoints accept a barcode/SKU as identifier; the scanner acts as keyboard input.
ACTORS	Warehouse operator

Performance, security, and operations

NFR-001 Performance

Simple reads return in under 300 ms under normal load; typical API responses complete in under 2 s for aggregated reports.

NFR-002 Security

All APIs require authenticated access; internal endpoints require the internal auth filter; the service is protected against OWASP Top 10 risks.

NFR-003 Availability

Target 99.9% monthly uptime.

NFR-004 Scalability

Application layer is horizontally scalable; database read replicas support reporting load.

NFR-005 Reliability & Consistency

ACID guarantees for stock transactions; database transactions and locking strategies enforce consistency.

NFR-006 Usability

Core operations — receive, ship, reserve — complete within three steps in the UI.

NFR-007 Audit & Retention

Transaction logs are retained per regulatory policy; configurable, default three years.

NFR-008 Backup & Recovery

> Nightly database backups with point-in-time recovery.

6 — USER ROLES & PERMISSIONS

Who can do what

Admin	Full system access — manage users, products, warehouses, run migrations.
Manager	Product and report access; set reorder levels.
Operator	Create transactions (receive/ship), perform stock counts.
Auditor	Read-only access to all logs.
Internal service	Machine client authenticating via API key or the internal auth filter.

Entities, relationships, validation, retention

7.1 Main Entities

Entity	Key fields
ProductEntity	id (UUID), sku, name, description, category, unit, priceTier1/2/3, stockQty (BigDecimal), reservedQty, status, created_at, updated_at
StockTransactionEntity	id, product_id, txn_type (TransactionType), qty, txn_date, reference, user_id, metadata
WarehouseEntity	id, name, address, metadata
Reservation	reservation_id, lines, status, created_at, fulfilled_at
User	id, username, email, role, status

7.2 Relationships

- Product → 1..* StockTransaction
- Warehouse holds stocks (future extension: a per-warehouse stock table)
- Reservation references product lines

7.3 Validation Rules

- SKU is unique and non-empty
- Quantities must be ≥ 0
- Price tiers must be non-negative
- Required fields are validated in DTOs

7.4 Data Retention

- Transaction logs: default 3 years, configurable

- Soft deletes for products/warehouses are permitted; audit history is preserved

8 — API / INTEGRATION REQUIREMENTS

Internal endpoints, third-party APIs, exports

8.1 Internal APIs (representative)

Method	Path
GET/POST	`/inventory/products`
GET/POST	`/inventory/transactions`
POST	`/internal/inventory/reservations/validate`
POST	`/internal/inventory/reservations/reserve`
POST	`/internal/inventory/reservations/fulfill`
GET/POST	`/inventory/warehouses`

8.2 Third-Party APIs

- ERP/POS — push/pull product and sales/order data
- Email/SMS gateway for alerts

8.3 Webhooks & Exports

- Webhook support for reservation/transaction events (recommended)
- CSV export for reports

9 — UI/UX REQUIREMENTS

Pages, responsiveness, forms

- **Pages:** Dashboard, Product List, Product Detail, Receive, Ship, Transfer, Reservation Validate/Reserve/Fulfill, Warehouses, Reports, Admin
- Responsive for desktop and tablet; barcode scanning supported as keyboard input
- Forms validate inline with clear error messages matching `ApiErrorResponse`

10 — SECURITY REQUIREMENTS

Authentication, authorization, encryption, audit

- **Authentication:** JWT tokens for users via `JwtAuthenticationFilter`
- **Authorization:** Role-based access control enforced in controllers/services
- **Encryption:** TLS in transit; at-rest encryption for sensitive fields as required
- **Password policy:** strong passwords required; optional SSO/OIDC/SAML
- **Audit trail:** immutable `stock_transactions` logs
- **Session/token management:** defined token expiry and refresh policy

11 — REPORTS & NOTIFICATIONS

What gets reported and who gets told

- **Standard reports:** stock level by SKU/location, movement history, reorder alerts
- **Exports:** CSV/PDF
- **Notifications:** low-stock alerts in the UI, with optional email

12 — BUSINESS RULES

Rules that govern stock movement

BR-001	A reservation must decrement available stock (`stockQty − reservedQty`).
BR-002	Fulfillment requires `reservedQty ≥ fulfill quantity` .
BR-003	Outbound transactions that would cause negative available stock must be rejected, unless an admin override is applied.
BR-004	Purchase orders and receipts update `stockQty` via the inbound transaction type.

13 — ERROR HANDLING

Standard error responses

Condition	Response
Validation errors	`400 BAD REQUEST`
Authentication failures	`401 UNAUTHORIZED`
Authorization failures	`403 FORBIDDEN`
Resource not found	`404 NOT FOUND`
Duplicate records	`409 CONFLICT`
Database/network failures	`5XX (WITH RETRY GUIDANCE)`

All error bodies conform to `ApiErrorResponse`.

14 — ACCEPTANCE CRITERIA

Definition of done

- Product CRUD works end-to-end with database persistence and migrations applied.
- Stock transactions update `product.stockQty` and are logged in `stock_transactions`.
- The reservation flow (validate → reserve → fulfill) prevents overcommit, with an atomic commit per request.
- Security filters block unauthorized access; internal endpoints accept only trusted clients.
- Integration endpoints return documented responses and error codes.

15 — CONSTRAINTS & DEPENDENCIES

Fixed platform decisions

- **Runtime:** Java + Spring Boot
- **Database:** PostgreSQL (connection string in `application.yml`)
- **Migrations:** Flyway scripts under `resources/db/migration`
- Must follow the repository package structure — `com.fawnix.inventory`

16 — FUTURE ENHANCEMENTS

Deferred, not forgotten

- Offline-capable mobile app for scanning and counts
- Forecasting and reorder automation
- Multi-warehouse per-product stock ledger
- Webhook/event stream (Kafka) for high-throughput integrations
- OpenAPI / Swagger documentation generation

APPENDIX A

Key code locations

Component	Path
ProductController	`src/main/java/com/fawnix/inventory/products/controller/ProductController.java`
Product service / repository / entity	`src/main/java/com/fawnix/inventory/products/`
Reservation controller / service	`src/main/java/com/fawnix/inventory/reservations/`
StockTransaction controller / service / entity	`src/main/java/com/fawnix/inventory/transactions/`
Warehouse modules	`src/main/java/com/fawnix/inventory/warehouses/`
Security filters and JWT	`src/main/java/com/fawnix/inventory/security/`

APPENDIX B

Database migrations

Migration	Contents
V2	`V2__create_inventory_schema.sql` — stock table and stock_transactions
V3–V6	Price tiers, reserved quantity, warehouses, seed catalog

APPENDIX C

Suggested next deliverables

1. Produce this SRS as a Markdown file and commit it to the repository
2. Generate OpenAPI (Swagger) documentation from controller DTOs
3. Create sequence diagrams for the reservation flow
4. Add an automated concurrency test for stock updates